

Debounce — JS Revision Notes

What is Debounce?

Debounce ensures a function runs only AFTER the event has stopped firing for a specified delay.

Common use cases:

- Search / autocomplete
- API calls while typing
- Form validation
- Resize events

Mental Model

Every event:

1. Clear previous timer
2. Start a new timer
3. If another event occurs → clear + restart
4. If no event occurs during **delay** → callback executes

type → clear → start timer type → clear → start timer type → clear → start timer stop typing ↓ delay completes ↓ callback executes

Basic Implementation

```
function debounce(callback, delay) { let timerID;
```

```
  return function (...args) {  
    clearTimeout(timerID);  
  
    timerID = setTimeout(() => {  
      callback(...args);  
    }, delay);  
  };  
}
```

Why Closure?

```
let timerID;
```

timerID belongs to the outer **debounce()** function.

The returned function closes over **timerID**, so the same timer reference is available across multiple calls.

Example:

```
debounced("A") ↓ timerID = timer1
```

```
debounced("AB") ↓ clear timer1 timerID = timer2
```

```
debounced("ABC") ↓ clear timer2 timerID = timer3
```

Only timer3 eventually executes.

Why clearTimeout()?

```
clearTimeout(timerID);
```

Cancels the previously scheduled callback.

This is what creates the "wait until the user stops" behavior.

Without clearTimeout():

type → timer1 type → timer2 type → timer3 ↓ all callbacks execute ❌

With clearTimeout():

type → timer1 type → cancel timer1 → timer2 type → cancel timer2 → timer3 stop → timer3 executes ✅

Why ...args?

```
return function (...args)
```

...args collects ALL arguments passed to the debounced function into an array.

Example:

```
debouncedInput("Hello");
```

```
args = ["Hello"]
```

Multiple arguments:

```
debounced(a, b, c);
```

```
args = [a, b, c]
```

Then:

```
callback(...args);
```

is equivalent to:

```
callback(a, b, c);
```

this — Is It Required?

NO.

For our current use case:

```
function updateOutput(value) { userOutput.textContent = value; }
```

`updateOutput()` does not use `this`.

So this is perfectly sufficient:

```
callback(...args);
```

Then Why Do Some Debounce Implementations Use `apply(this, args)`?

A generic debounce utility may need to preserve the `this` context of the callback.

Example:

```
const user = { name: "Avishek",
```

```
  greet() {  
    console.log(this.name);  
  }  
};
```

Here `greet()` depends on:

`this` → `user`

In such reusable utilities, you may see:

```
callback.apply(this, args);
```

Meaning:

```
callback.apply(thisArg, argumentsArray);
```

- First argument → what `this` should refer to
- Second argument → arguments to pass to callback

`apply()` Mental Model

```
callback.apply(thisValue, argsArray);
```

Example:

```
function greet(name, age) { console.log(this.city, name, age); }
```

```
greet.apply( { city: "Gurgaon" }, ["Avishek", 25] );
```

Conceptually:

`this` → `{ city: "Gurgaon" }` `args` → `["Avishek", 25]`

Equivalent to:

```
greet("Avishek", 25);
```

but with:

```
this → { city: "Gurgaon" }
```

call vs apply vs bind

```
call: callback.call(thisValue, value1, value2);
```

→ arguments individually

```
apply: callback.apply(thisValue, [value1, value2]);
```

→ arguments as an array

```
bind: const fn = callback.bind(thisValue);
```

→ returns a NEW function

Remember:

call → individual arguments apply → array of arguments bind → returns function

Important Distinction

These are TWO separate concepts:

...args ↓ What arguments should callback receive?

this ↓ What should **this** refer to?

Therefore:

```
callback.apply(this, args);
```

means:

"Call callback with this context and pass args as arguments."

For Our Current Debounce

We don't need **this**.

Use:

```
function debounce(callback, delay) { let timerID;
```

```
  return function (...args) {  
    clearTimeout(timerID);  
  
    timerID = setTimeout(() => {
```

```
        callback(...args);  
    }, delay);  
};
```

```
}
```

Common Mistake #1

Wrong:

```
callback.apply(this, ...args); ❌
```

Correct:

```
callback.apply(this, args); ✅
```

Because `apply()` expects:

```
apply(thisValue, argsArray)
```

Common Mistake #2

Wrong:

```
return function (args) { ... };
```

This means `args` is ONE argument.

Preferred:

```
return function (...args) { ... };
```

This collects all arguments into an array.

Common Mistake #3 — Calling Debounced Function Twice

Wrong:

```
input.addEventListener("input", (event) => { console.log(debouncedInput(event.target.value));  
debouncedInput(event.target.value); });
```

This calls the debounced function twice.

Correct:

```
input.addEventListener("input", (event) => { debouncedInput(event.target.value); });
```

Interview Mental Model

Debounce = "Wait until activity stops."

Closure ↓ remembers timerID

`clearTimeout()` ↓ cancels previous scheduled execution

`setTimeout()` ↓ schedules new execution

`...args` ↓ captures arguments

`callback(...args)` ↓ executes callback with those arguments

`this` ↓ ONLY needed when callback depends on its `this` context

Interview One-Liner

"Debounce delays function execution until the event has stopped firing for a specified period. It uses a closure to retain the timer ID, clears the previous timer, and schedules a new execution."